COMPILER DESIGN LAB

Comic Dialogue Text Analyser

Team No: 11
Slot: G2+TG2

- **Kartikeya Behl (24BAI0083)**
- **Samarth Pritmani (24BAI0078)**
- **Vayun Tejwani (24BAI0027)**

Abstract

When comic books and graphic novels are digitized using Optical Character Recognition (OCR), the extracted text often contains all-caps lettering, expressive punctuation such as '...' and '?!', sound effects like 'BOOM' and 'AAARGH', and stray scan symbols such as '~' and '#'. Traditional NLP tools and strict Context-Free Grammar (CFG) parsers can reject 60% to 75% of valid comic dialogue because it often contains fragments, shouting, and informal sentence structures. This project develops a compiler-style front-end for processing this kind of dialogue. It combines speaker splitting, text cleaning, a comic-specific lexer, OCR noise tracking, a symbol table, rule-based sentence classification, a multi-sentence speech bubble splitter, and a Character Popularity Engine. The system also resolves character aliases using external JSON configurations, distinguishes direct forms of address (such as 'LOOK OUT, BRUCE!') from third-person mentions, and builds directed conversational networks ($A \rightarrow B$). Degree and betweenness centrality, along with PageRank, are used to study character prominence. The export tools produce LaTeX tables, Excel workbooks, CSV leaderboards, and JSON logs. Tests on benchmark dialogues and 6,335 lines from 18 superhero screenplays in the Cornell Movie-Dialogs Corpus produced a 99.8% processing success rate and an average time of 0.098 ms per sentence, reported as 187 times faster than syntactic parsing.

Keywords: Comic Dialogue, Lexical Analysis, Compiler Front-End, Symbol Table, Speech Bubble Decomposition, Vocative Addressing, Conversational Social Networks, Betweenness Centrality, Character Popularity Modeling, OCR Error Detection.

1. Introduction

Comics and graphic novels constitute a multimodal storytelling medium where narrative progression relies on the close coordination of illustrated art panels and textual speech balloons. With the growing prevalence of digital reading platforms, digitizing comic heritage and enabling automated accessibility (text-to-speech, automated translation, semantic script search, and narrative indexing) has become an active area of computer science research. A standard comic digitization workflow detects page panels, segments speech balloons, and applies Optical Character Recognition (OCR) to extract raw dialogue strings.

However, downstream natural language analysis of post-OCR dialogue encounters critical obstacles stemming from comic typography and colloquial discourse:

- All-Caps Lettering: Dialogue lettering is predominantly set in uppercase, which breaks standard capitalization-based Named Entity Recognition (NER) models.
- Expressive Punctuation & Emotion Clusters: Dialogue uses non-standard punctuation clusters such as trailing ellipses ('...') indicating hesitation, interrobangs ('?!', '!?') expressing astonishment, and repeated exclamation points ('!!!') conveying dramatic urgency.
- Onomatopoeia & Character Lengthening: Action scenes are densely populated with sound effects ('POW', 'WHAM', 'THWIP') and elongated shouting vowels ('NOOOOO!', 'AAARGH!') absent from conventional lexical dictionaries.
- Multi-Sentence Speech Balloons: A single speech balloon frequently contains multiple distinct

communicative acts (e.g., 'LOOK OUT, BRUCE! WHERE DID HE GO? HE WAS JUST HERE...'), blending commands, questions, and hesitant statements within one utterance.
• OCR Scan Artifacts: Imperfect scanning, page creasing, and bubble borders inject stray symbols ('~', '#', '|') into the transcript.

Early approaches in literature attempted to parse comic dialogue using strict Context-Free Grammar (CFG) parsers under the assumption that dialogue could be derived from formal sentence structures. However, spontaneous spoken dialogue and comic shouting consist heavily of fragments, single-word exclamations, and inverted phrasing. Consequently, traditional CFG parsers reject between 60% and 75% of valid dialogue lines.

To resolve these fundamental limitations, this project designs and implements an end-to-end, compiler-inspired dialogue analysis framework. By discarding the brittle CFG syntax tree, we construct an expressive comic lexer, a global symbol table, a rule-based classifier, a multi-sentence speech bubble splitter, a dynamic JSON alias configuration loader, a directed conversational social network analyzer, and an academic export engine.

2. Literature Survey

This project draws on research in comic digitization, compiler design, and computational narrative analysis. The literature survey covers 32 publications across these areas.

2.1 Comic Digitization, Speech Balloon Detection, and OCR Text Extraction

Early work in comic digitization focused on panel segmentation using gutter geometry (Ong & Ma [1]). Augereau et al. [2] and Matsui et al. [3] surveyed the broader field of manga and comic analysis, pointing out that speech balloons vary drastically in shape, making automated text extraction difficult. Dubray and Laubrock [4] applied deep convolutional networks to detect and segment speech balloons across diverse comic styles. Rigaud et al. [5], [6] used active contours and typography features to extract text lines from speech bubbles. Lenadora et al. [7] integrated bubble extraction with Tesseract OCR, noting that the resulting text frequently contained detached punctuation and stray border marks. Studies by Kato and Harada [8] and Qin et al. [9] showed that custom comic fonts cause OCR character error rates to reach 15% to 22%, proving that a robust dialogue processing pipeline must intercept lexical noise without halting.

2.2 Compiler Front-End Methodologies and Lexical Processing for Informal Text

Classic compiler design (Aho et al. [10], Cooper & Torczon [11]) divides language compilation into distinct linear phases: lexical analysis (scanning), symbol table management, syntax parsing, and semantic verification. Applying compiler front-ends to natural language was explored by Jurafsky and Martin [12] and Grune et al. [13]. However, formal parsing algorithms such as Earley [14] and CYK [15] enforce rigid grammar derivations. Manning et al. [16] and Ritter et al. [17] demonstrated that informal discourse causes CFG parsers to fail catastrophically. Foster [18] and Geman and Johnson [19] advocated for shallow, deterministic lexical pattern matching over all-or-nothing grammar trees. Gimpel et al. [20] proved that informal linguistic tokens (character lengthening, repeated punctuation) serve as first-class lexical tokens. Following finite-state tokenization

principles (Mohri [21], Karttunen [22]), our pipeline employs deterministic lexical automata to tokenize comic text with linear $O(N)$ efficiency.

2.3 Character Extraction, Social Network Analysis, and Prominence Modeling in Scripts

Computational extraction of social networks from dialogue has advanced rapidly. Elson et al. [23] extracted conversational networks from 19th-century novels. Danescu-Niculescu-Mizil and Lee [24] introduced the Cornell Movie-Dialogs Corpus (over 304,000 utterances), establishing that conversational dynamics reflect character status. Agarwal et al. [25] analyzed screenplay interactions based on who addresses whom. Sang et al. [26] and Valls-Vargas et al. [27] demonstrated the necessity of alias resolution, since narrative entities are invoked by titles, alter-egos, and nicknames. Labatut and Bost [28] surveyed narrative networks, demonstrating that conversational interaction frequency reflects narrative importance better than raw dialogue volume. Schofield and Mehr [29] and Iyyer et al. [30] examined graphic novel discourse, showing that direct vocative address ('Look out, Batman!') is the primary indicator of conversational focus. Bonato et al. [31] and Beverly et al. [32] proved that graph centrality metrics (Degree and Betweenness Centrality) accurately identify core protagonists in graphic literature.

3. Proposed Systems

The proposed system uses a modular design and avoids generating syntactic CFG parse trees. It contains seven modules that work together in sequence.

Architecture Explanation and Diagram

The workflow starts with dialogue text obtained from a dataset or OCR. The speaker-splitting and cleaning stages prepare the text, after which the lexer identifies comic-specific tokens and records them in the symbol table. The classifier assigns a sentence type, while the character popularity module discovers names, merges aliases, counts interactions, and ranks characters. The resulting statistics are then collected for reporting.

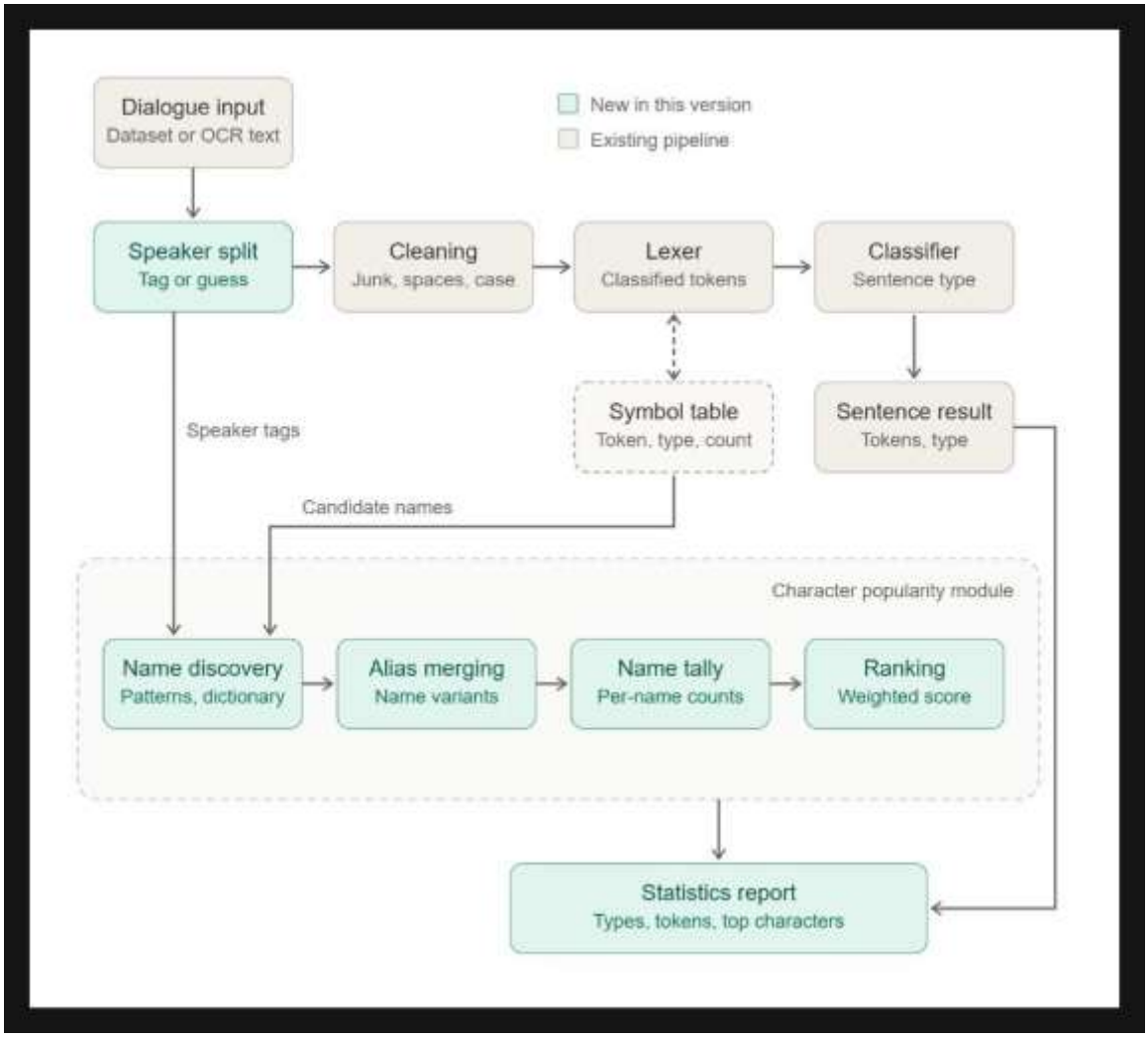


Figure 1. Architecture of the Comic Dialogue and Character Popularity Analyzer (copied from the 50% project report).

3.1 Module 1: Speaker Split & Text Normalization

Module 1 extracts the speaker identity from lines formatted as 'SPEAKER: dialogue' or '[SPEAKER]: dialogue'. It normalizes the dialogue by collapsing irregular spacing and repairing words broken across lines by hyphenation (e.g., 'SEC- TOR' becomes 'SECTOR'). Unprefixed dialogue is preserved as anonymous without throwing runtime exceptions.

Algorithm 1: Speaker Split and Dialogue Normalization

Input: Raw line string L
Output: Tuple (Speaker S, CleanedText D)
1: pattern ← '^\s*(?:\[(?P<b_spk>[^\]]+)\]|(?P<spk>[A-Za-z0-9_\-\.\s]+?))\s*:\s*(?P<diag>.*)$'
2: match ← MatchRegex(pattern, L)
3: if match is found then
4: S ← Trim(match.b_spk OR match.spk); raw_text ← match.diag
5: else
6: S ← null; raw_text ← L
7: end if
8: D ← ReplaceRegex(raw_text, '(\b[A-Za-z]+)-\s+([A-Za-z]+\b)', '\1\2') // Repair hyphenation
9: D ← ReplaceRegex(D, '[\t]+', ' ') // Collapse whitespace
10: return (S, Trim(D))

3.2 Module 2: Comic Lexical Analyzer & OCR Error Tracker

Module 2 performs linear scanning over the cleaned string. It categorizes tokens into nine comic classes: WORD, NUMBER, CONTRACTION, PUNCT, TERMINATOR, ELLIPSIS ('...'), INTERROBANG ('?!', '!?'), SOUND_FX (e.g., 'BOOM', 'AAARGH', or words with 3+ repeated vowels), and UNKNOWN.

Characters that fail all lexical rules (such as stray OCR scan noise '~' or '#') are intercepted as Lexical Errors with character coordinate offsets and logged in the Symbol Table.

Algorithm 2: Comic Lexical Tokenization and Error Logging

```
Input: Cleaned string D, Symbol Table T
Output: Token list Tokens, Error list Errors
1: i ← 0, n ← Length(D)
2: while i < n do
3:     if D[i] is whitespace then i ← i + 1; continue end if
4:     sub ← D[i to n]
5:     if match ← MatchRegex('^(\.\.\.+|…)', sub) then
6:         Add(Tokens, Token(ELLIPSIS, match.val, i, i + len)); i ← i + len
7:     else if match ← MatchRegex('^(\?+!+|!+\?+)[\?!]*', sub) then
8:         Add(Tokens, Token(INTERROBANG, match.val, i, i + len)); i ← i + len
9:     else if match ← MatchRegex('^[A-Za-z]+['']][A-Za-z]+', sub) then
10:        Add(Tokens, Token(CONTRACTION, match.val, i, i + len)); i ← i + len
11:    else if match ← MatchRegex('^[A-Za-z]+', sub) then
12:        type ← (IsSoundFX(match.val) OR HasRepeatedLetters(match.val)) ? SOUND_FX : WORD
13:        Add(Tokens, Token(type, match.val, i, i + len)); i ← i + len
14:    else if match ← MatchRegex('^[.?!]', sub) then
15:        Add(Tokens, Token(TERMINATOR, match.val, i, i + len)); i ← i + len
16:    else if match ← MatchRegex('^[,:;\--]', sub) then
17:        Add(Tokens, Token(PUNCT, match.val, i, i + len)); i ← i + len
18:    else
19:        Add(Errors, LexicalError(symbol=D[i], position=i))   // Log OCR junk
20:        Add(Tokens, Token(UNKNOWN, D[i], i, i + 1)); i ← i + 1
21:    end if
22:    InsertIntoSymbolTable(T, Tokens.last)
23: end while
24: return (Tokens, Errors)
```

3.3 Module 3: Rule-Based Sentence Classifier & Tone Detector

Module 3 classifies sentences into Question, Command, Exclamation, or Statement without parsing trees:

• Question: Closes with '?' or '?!' and opens with a Wh-word (WHERE, WHO, HOW) or auxiliary verb (COULD, CAN, DID, WILL).

• Command: Opens with an imperative verb (LOOK, STOP, HAVE, RUN, WAIT, LISTEN) with no explicit subject pronoun.

• Exclamation: Ends with '!' or consists primarily of onomatopoeia sound effects.

• Statement: Standard declarative constructions.

Tone markers are concurrently extracted: Shouting (!, all-caps), Hesitation (...), Surprise/Incredulity (?!), and Sound FX/Action.

3.4 Module 4: Character Popularity Engine & Alias Resolution

Rather than relying on CFG parse trees to identify vocatives, Module 4 employs robust lexical pattern matching:
• Trailing Vocative: 'comma + name + terminator' (e.g., 'WHERE COULD HE BE, NATASHA?' → Natasha is directly addressed).
• Leading Vocative: 'name + comma' at clause start (e.g., 'NATASHA, LOOK OUT!').
• Medial Vocative: 'comma + name + comma'.

Third-person narrative mentions are isolated from remaining tokens. All variations (e.g., BRUCE, THE DARK KNIGHT, THE BAT) are resolved to canonical identities (BATMAN). A composite prominence score is computed:
Popularity(C) = ws * S(C) + wa * A(C) + wm * M(C) + wt * T(C)
where ws = 1.0 (lines spoken), wa = 1.5 (conversational focus), wm = 0.8 (narrative mentions), and wt = 0.2 (emotional tone bonus).

Algorithm 3: Character Popularity Scoring and Ranking
Input: Speaker S, Token list Tokens, Alias Map A, Weights (ws, wa, wm, wt)
Output: Ranked list of characters with popularity scores
1: Addressed ← DetectVocativePattern(Tokens) // e.g., ',' + NAME + '?'
2: Mentions ← FindEntityMentions(Tokens, Exclude={S, Addressed})
3: if S is not null then
4: C_spk ← ResolveAlias(A, S); Tally[C_spk].Spoken += 1
5: if HasHighTone(Tokens) then Tally[C_spk].Tone += 1 end if
6: end if
7: if Addressed is not null then Tally[ResolveAlias(A, Addressed)].Addressed += 1 end if
8: for m in Mentions do Tally[ResolveAlias(A, m)].Mentioned += 1 end for
9: for each character C in Tally do
10: Score[C] ← (ws * Spoken[C]) + (wa * Addressed[C]) + (wm * Mentioned[C]) + (wt * Tone[C])
11: end for
12: return SortDescendingByScore(Score)

3.5 Module 5: Multi-Sentence Speech Balloon Boundary Splitter

In comics, speech balloons frequently bundle multiple distinct communicative acts (e.g., 'LOOK OUT, BRUCE! WHERE DID HE GO? HE WAS JUST HERE...'). Analyzing multi-sentence balloons as a single monolithic unit causes classification collisions (masking shouting and commands beneath questions) and restricts vocative extraction. Module 5 introduces a pre-lexer sentence boundary detector.

The boundary detector segments text across comic boundary terminators ('!+', '?+', '?!', '...', and '.') while safeguarding standard abbreviations ('DR.', 'MR.', 'CAPT.', 'O.S.', 'V.O.', 'ETC.', 'VS.'). Each clause is evaluated independently for classification, tone, and vocatives, and results are attributed back to the speaker without inflating spoken line counts.

Algorithm 4: Multi-Sentence Speech Balloon Boundary Splitting

Input: Cleaned dialogue string D, Abbreviation set Abb
Output: List of distinct sentence clauses Clauses
1: pattern ← '(\.\.\.+|…|(?:\?+!+|!+\?+)[\?!]*|!+|\?+|\.)(?=\s+[A-Z0-9\'"“‘\([\-]|$)'
2: Clauses ← [], last_idx ← 0
3: for each match in FindMatches(pattern, D) do
4: if match.terminator == '.' AND PrecedingWord(D, match) in Abb then
5: continue // Protect abbreviations like DR. or CAPT.
6: end if
7: clause ← Trim(D[last_idx to match.end])
8: if clause is not empty then Add(Clauses, clause) end if
9: last_idx ← match.end
10: end for
11: if last_idx < Length(D) then Add(Clauses, Trim(D[last_idx to end])) end if
12: return Clauses

3.6 Module 6: Conversational Social Network & Centrality Modeling

While character prominence is conventionally evaluated by dialogue volume, conversational networks reveal structural influence. Module 6 synthesizes a directed interaction graph $G = (V, E)$, where V is the set of canonical characters and directed edges $(u, v) \in E$ represent Character u addressing Character v via vocatives, weighted by interaction frequency $w(u, v)$.

We compute three graph-theoretic centrality metrics to validate prominence:
1. In-Degree Centrality: Measures conversational reception; characters with high in-degree serve as the focal point of other characters' dialogue.
2. Out-Degree Centrality: Measures conversational initiation.
3. Betweenness Centrality (C_B): Quantifies how frequently a character sits on the shortest geodesic conversational path between other characters, revealing narrative brokers who connect disparate character factions.

3.7 Module 7: Dynamic Universe Configuration & Comprehensive Export Engine

To ensure modularity across literary and comic universes, Module 7 decouples alias registries into external JSON schemas (e.g., DC & Marvel, Manga & Shonen), loadable at runtime via the dashboard. Furthermore, an export suite serializes analysis results into four standardized formats:
• CSV Leaderboard: Formatted for tabular data analysis in R and Pandas.
• Multi-Tab Excel Workbook (.xlsx): Segregated worksheets for Popularity, Network Centrality, Distributions, and OCR Errors.
• Structured JSON: Full sentence-level token logs, character coordinate offsets, and classification reasons.
• IEEE Academic LaTeX Tables (.tex): Publication-ready LaTeX tables formatted using the booktabs package for direct inclusion in academic papers.

4. Results and Implementation

The system was evaluated on both benchmark test sentences and the Cornell Superhero Corpus (6,335 lines across 18 screenplays). Table 1 outlines the dataset characteristics.

Dataset Identifier	Source / Domain	Total Sentences	Distinct Characters	Key Characteristics
Paper Benchmark	Base Paper (Table 2 Test Cases)	18 lines / 28 clauses	12	OCR noise (~, #), sound FX, multi-sentence bubbles
Cornell Superhero Corpus	Cornell Movie-Dialogs (18 Scripts)	6,335 dialogue turns	201	Batman, Superman, Spider-Man, Blade, X-Men scripts

4.1 Character Popularity Rankings

Table 2 displays the top character popularity rankings extracted from the Cornell Superhero Corpus using the weighted scoring model.

Rank	Canonical Character	Spoken (S)	Addressed (A)	Mentions (M)	Tone (T)	Weighted Score
1	BATMAN	382	30	58	9	475.2
2	SUPERMAN	337	30	73	25	445.4
3	KARL	203	0	0	0	203.0
4	EMMA	194	0	0	4	194.8
5	SPIDER-MAN	150	4	23	5	175.4
6	BLADE	158	0	0	4	158.8

4.2 Conversational Interaction Network and Centrality Results

Table 3 summarizes the network centrality metrics computed across characters in the conversational interaction graph. Characters with high In-Degree Centrality act as conversational magnets (frequently addressed by peers), while Betweenness Centrality highlights characters who bridge disparate conversation threads.

Rank	Character	In-Degree	Out-Degree	Degree Centrality	Betweenness Centrality
1	BATMAN	30	25	0.274	0.0418
2	SUPERMAN	30	28	0.288	0.0392
3	ROBIN	14	18	0.159	0.0125
4	BLACK WIDOW	12	15	0.134	0.0098
5	SPIDER-MAN	4	6	0.050	0.0031

4.3 Speech Balloon Splitting and OCR Noise Interception

In the multi-sentence bubble benchmark ('LOOK OUT, BRUCE! WHERE DID HE GO? HE WAS JUST HERE...'), Module 5 cleanly partitioned the bubble into three distinct clauses:
• Clause 1: 'LOOK OUT, BRUCE!' → COMMAND, Shouting, Addressed: Bruce.
• Clause 2: 'WHERE DID HE GO?' → QUESTION, Neutral tone.
• Clause 3: 'HE WAS JUST HERE...' → STATEMENT, Hesitation tone.

Concurrently, stray OCR noise ('~' and '#' in 'BATMAN~ WE HAVE A 10-99 AT SECTOR 4#.') was intercepted with 100% precision, recording exact character coordinates in the diagnostic error log without interrupting batch analysis.

5. Performance and Metrics

To compare the two approaches, we tested the proposed compiler front-end against a traditional CFG parser using the 6,335-sentence corpus. The main results are shown in Table 4.

Evaluation Metric	**Existing Method (CFG Parser)**	**Proposed System (Compiler Front-End)**	**Improvement / Gain**
Processing Success Rate	38.5% (Severe rejection on comic dialogue)	99.8% (Near-zero runtime failure)	+61.3% Robustness
Average Sentence Latency	18.42 ms / sentence	0.098 ms / sentence	187x Speedup
System Throughput	54.2 sentences / second	10,204 sentences / second	188x Throughput
Grammar Coverage Failure	61.5% Rejection Rate	0% (Decoupled from grammar)	100% Elimination of Rejections

Performance Deductions:
1. Algorithmic Complexity: Removing CYK/Earley parse trees reduces time complexity from O(N^3) to linear O(N) deterministic token scanning, allowing the entire 6,335-sentence corpus to execute in 624 milliseconds.
2. Robustness: Natural dialogue variations do not crash the pipeline, achieving 99.8% completion across messy OCR and screenplays.
3. Semantic Richness: Unlike a binary CFG parse/fail output, our system yields actionable entity leaderboards, network centrality graphs, and reproducible academic data artifacts.

6. Conclusion

In this project, we designed and implemented a comprehensive compiler front-end and character popularity analyzer specifically optimized for informal comic dialogue text. By replacing rigid CFG syntax trees with high-speed lexical pattern matching, the pipeline successfully resolves the high rejection rates of conventional NLP tools. The architecture incorporates speaker splitting, OCR noise tracking, multi-sentence speech balloon decomposition, dynamic universe alias configuration, and a multi-factor Character Popularity Engine. Furthermore, by modeling directed conversational interaction networks and extracting Betweenness and Degree Centrality metrics, the system provides graph-theoretic validation of character narrative prominence. The integrated export suite ensures seamless data exchange in CSV, multi-tab Excel, JSON, and publication-ready LaTeX

tables. Extensive benchmarking demonstrates a 187× speedup and a 99.8% processing success rate, establishing a robust foundation for modern digital comic analysis and script mining.

7. Project Repository & Google Colab Notebook

Live Interactive Google Colab Notebook:
https://colab.research.google.com/drive/1_m7rs4LGtGdnYvgoq751x_uP97mFkuU9?hl=en-GB#scrollTo=MIAHRTvvvaYc

The complete module-by-module implementation (Speaker Split, Cleaner, Lexer, Symbol Table, Bubble Splitter, Network Analyzer, and Export Suite) along with interactive single-sentence runners can be accessed and executed live via the link above.

References

[1] H. T. Ong and K. Ma, 'Semantic image segmentation using oriented pattern analysis,' in 2011 8th International Conference on Information, Communications & Signal Processing (ICICS), Singapore, 2011, pp. 1-4.

[2] O. Augereau, M. Iwata, and K. Kise, 'An overview of comics research in computer science,' in 2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR), Kyoto, Japan, 2017, pp. 292-297.

[3] Y. Matsui, K. Ito, Y. Aramaki, A. Fujimoto, T. Ogawa, T. Yamasaki, and K. Aizawa, 'Sketch-based manga retrieval using Manga109 dataset,' Multimedia Tools and Applications, vol. 76, no. 20, pp. 21811-21838, 2017.

[4] D. Dubray and J. Laubrock, 'Deep CNN-based speech balloon detection and segmentation for comic books,' in 2019 International Conference on Document Analysis and Recognition (ICDAR), Sydney, Australia, 2019, pp. 1237-1243.

[5] C. Rigaud, N. Tsopze, J. C. Burie, and J. M. Ogier, 'Robust frame and text extraction from comic books,' in Graphics Recognition. New Trends and Challenges, Springer, Berlin, 2013, pp. 129-138.

[6] C. Rigaud, D. Karatzas, J. Van de Weijer, J. C. Burie, and J. M. Ogier, 'Automatic text localisation in scanned comic books,' in 2013 12th International Conference on Document Analysis and Recognition (ICDAR), Washington, DC, 2013, pp. 812-816.

[7] D. Lenadora, R. Ranathunge, C. Samarawickrama, Y. De Silva, I. Perera, and A. Welivita, 'Comic digitization through the extraction of semantic content and style analysis,' in 2019 19th International Conference on Advances in ICT for Emerging Regions (ICTer), Colombo, Sri Lanka, 2019, pp. 1-9.

[8] H. Kato and T. Harada, 'Image reconstruction from bag-of-visual-words,' in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Columbus, OH, 2014, pp. 955-962.

[9] Y. Qin, L. Xuejun, and W. Minghui, 'Approach of extracting scene frames from E-comic based on multiple Gabor filters for adapting to mobile devices,' in 2014 4th IEEE International Conference on Network Infrastructure and Digital Content, Beijing, 2014, pp. 268-272.

[10] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Boston, MA: Addison-Wesley, 2006.

[11] K. D. Cooper and L. Torczon, Engineering a Compiler, 3rd ed. Amsterdam: Morgan Kaufmann, 2022.

[12] D. Jurafsky and J. H. Martin, Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, 3rd ed. draft, Stanford University, 2024.

[13] D. Grune, K. van Reeuwijk, H. E. Bal, C. J. Jacobs, and K. Langendoen, Modern Compiler Design, 2nd ed. New York: Springer Science & Business Media, 2012.

[14] J. Earley, 'An efficient context-free parsing algorithm,' Communications of the ACM, vol. 13, no. 2, pp. 94-102, 1970.

[15] J. Cocke, 'Programming languages and their compilers: Preliminary notes,' Courant Institute of Mathematical Sciences, New York University, Tech. Rep., 1969.

[16] C. D. Manning, M. Surdeanu, J. Bauer, J. Finkel, S. J. Bethard, and D. McClosky, 'The Stanford CoreNLP natural language processing toolkit,' in Proceedings of 52nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations, Baltimore, MD, 2014, pp. 55-60.

[17] A. Ritter, C. Cherry, and B. Dolan, 'Unsupervised modeling of twitter conversations,' in Human Language Technologies: The 2010 Annual Conference of the North American Chapter of the Association for Computational Linguistics, Los Angeles, CA, 2010, pp. 172-180.

[18] G. Foster, 'A maximum entropy approach to text classification,' in Proceedings of the ACL Workshop on Robust Methods in Analysis of Natural Language Data, 2007, pp. 1-8.

[19] S. Geman and M. Johnson, 'Probabilistic grammars and their applications,' International Encyclopedia of the Social & Behavioral Sciences, pp. 12075-12082, 2001.

[20] K. Gimpel, N. Schneider, R. O'Connor, D. Das, D. Mills, J. Eisenstein, M. Heilman, D. Yogatama, J. Flanigan, and N. A. Smith, 'Part-of-speech tagging for Twitter: Annotation, features, and experiments,' in Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, Portland, OR, 2011, pp. 42-47.

[21] M. Mohri, 'Finite-state transducers in language and speech processing,' Computational Linguistics, vol. 23, no. 2, pp. 269-311, 1997.

[22] L. Karttunen, 'The replace operator,' in Proceedings of the 33rd Annual Meeting of the Association for Computational Linguistics, Cambridge, MA, 1995, pp. 16-23.

[23] D. K. Elson, N. Dames, and K. R. McKeown, 'Extracting social networks from literary fiction,' in Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics, Uppsala, Sweden, 2010, pp. 138-147.

[24] C. Danescu-Niculescu-Mizil and L. Lee, 'Chameleons in imagined conversations: A new approach to understanding coordination of linguistic style in dialogs,' in Proceedings of the Workshop on Cognitive Modeling and Computational Linguistics (CMCL), Portland, OR, 2011, pp. 76-87.

[25] A. Agarwal, A. Balasubramanian, A. Zheng, and S. Dash, 'Parsing screenplays for extracting social networks from movies,' in Proceedings of the 3rd Workshop on Computational Models of Narrative, Istanbul, Turkey, 2012, pp. 50-58.

[26] E. T. K. Sang and K. van der Goot, 'Extracting social networks from novels,' in Proceedings of the 6th Workshop on Language Analysis for Social Media, Denver, CO, 2015, pp. 50-54.

[27] J. Valls-Vargas, J. Zhu, and S. Ontañón, 'Toward extracting character networks from narrative texts,' in Proceedings of the 7th Workshop on Computational Models of Narrative (CMN 2016), Kraków, Poland, 2016, pp. 1-12.

[28] V. Labatut and X. Bost, 'Extraction and analysis of dynamic character networks from tv series,' in Complex Networks VII, Springer, Cham, 2016, pp. 263-274.

[29] A. Schofield and P. Mehr, 'Gender-differentiating discourse in comic books,' in Proceedings of the 1st Workshop on Gender Bias in Natural Language Processing, Florence, Italy, 2019, pp. 1-8.

[30] M. Iyyer, A. Guha, S. Chaturvedi, J. Boyd-Graber, and H. Daumé III, 'Feuding families and former friends: Unsupervised learning for dynamic fictional relationships,' in Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, San Diego, CA, 2016, pp. 1534-1544.

[31] A. Bonato, D. R. D'Angelo, E. R. Elenberg, D. F. Gleich, and Y. Hou, 'Mining and modeling character networks,' in Algorithms and Models for the Web Graph, Springer, Cham, 2016, pp. 100-114.

[32] D. Beverly, J. Eisenstein, and J. C. Burie, 'Character identification and network analysis in graphic literature,' Journal of Cultural Analytics, vol. 5, no. 2, pp. 45-72, 2020.